Module-1

1. Define Data structures. Explain the classification of data structures with examples or Various operations of data structures or explain primitive and non primitive operations of data structures

Definition: A data structure is a specialized format for organizing, processing, retrieving, and storing data. It defines a way to arrange data in a computer's memory so that it can be used efficiently.

Classification of Data Structures:
Data structures are broadly classified into two categories:

1. **Primitive Data Structures:** These are the basic data structures directly operated upon by machine-level instructions.
    - **Examples:** int, float, char, double, pointer.
2. **Non-Primitive Data Structures:** These are more complex data structures derived from primitive ones. They are further divided:
    - **Linear Data Structures:** Elements form a sequence or a linear list.
        - *Static:* Memory allocation is fixed at compile time (e.g., **Array**).
        - *Dynamic:* Memory allocation is flexible at runtime (e.g., **Linked List**, **Stack**, **Queue**).
    - **Non-Linear Data Structures:** Elements do not form a sequence; a single element can be connected to multiple elements.
        - **Examples: Trees**, **Graphs**.
    - **File Structures:** Represents data in secondary storage (e.g., **Sequential File**, **Indexed File**).

Operations on Data Structures:

- **Primitive Operations:** Basic operations inherent to the data type (e.g., +, -, *, / for integers, assignment =).
- **Non-Primitive Operations:** Operations performed on the data structure itself.
    - **Traversal:** Accessing each element exactly once.
    - **Insertion:** Adding a new element.
    - **Deletion:** Removing an existing element.
    - **Searching:** Finding the location of a given element.

- **Sorting:** Arranging elements in a specific (ascending/descending) order.
- **Merging:** Combining two similar data structures into one.

---

2. Differentiate between static and dynamic memory allocations. Discuss four dynamic memory allocation functions supported by C with syntax and examples

Feature	Static Memory Allocation	Dynamic Memory Allocation
Time	At compile time	At run time
Size	Fixed, cannot be changed	Flexible, can be resized
Memory	From Stack	From Heap
Efficiency	Faster access	Slower access
Control	Managed by compiler	Managed by programmer
Functions	Not applicable	malloc(), calloc(), realloc(), free()

Dynamic Memory Allocation Functions in C:

1. malloc() **(Memory Allocation):** Allocates a single block of memory of a specified size. It does not initialize the memory.
    - **Syntax:** ptr = (cast-type*) malloc(byte-size);
    - **Example:**

c

int *ptr;

ptr = (int*) malloc(100 * sizeof(int)); *// Allocates memory for 100 integers*

2. calloc() **(Contiguous Allocation):** Allocates multiple blocks of memory, initializes all bits to zero.
    - **Syntax:** ptr = (cast-type*) calloc(n, element-size);
    - **Example:**

c

```
int *ptr;
ptr = (int*) calloc(100, sizeof(int)); // Allocates & zeros memory for 100 integers
```

3. realloc() **(Re-allocation):** Changes the size of a previously allocated memory block.
    - **Syntax:** ptr = realloc(ptr, new-size);
    - **Example:**

c

```
ptr = (int*) realloc(ptr, 150 * sizeof(int)); // Resizes the array to 150 integers
```

4. free()**:** De-allocates the dynamically allocated memory to prevent memory leaks.
    - **Syntax:** free(ptr);
    - **Example:**

c

```
free(ptr); // Frees the allocated memory
ptr = NULL; // Good practice to avoid dangling pointer
```

---

3. Define pointers. How to declare and initialize pointers, explain with example. List the Advantages of pointers over arrays and explain void pointers

Definition: A pointer is a variable that stores the memory address of another variable.

Declaration and Initialization:

- **Declaration:** data_type *pointer_name;
- **Initialization:** pointer_name = &variable_name;
- **Example:**

c

```
int x = 10;
int *ptr;     // Declaration
ptr = &x;     // Initialization, ptr now holds the address of x

printf("Value of x: %d\n", x);     // Output: 10
printf("Address of x: %p\n", &x);   // Output: address of x
```

printf("Value of ptr: %p\n", ptr); *// Output: same as &x*

printf("Value at ptr: %d\n", *ptr); *// Output: 10 (Dereferencing)*

Advantages of Pointers over Arrays:

- **Dynamic Memory Allocation:** Pointers are essential for dynamic data structures (linked lists, trees) where memory is allocated at runtime.
- **Pass by Reference:** Functions can modify the original variables passed to them using pointers.
- **Efficiency:** Passing a pointer to a large structure is more efficient than passing the entire structure by value.
- **Memory Efficiency:** Can create complex data structures without wasting memory.
- **Return Multiple Values:** A function can return multiple values by modifying variables via pointers.

Void Pointers (Generic Pointers):
A void pointer (void*) is a special pointer that can hold the address of any data type. However, it cannot be dereferenced directly; it must be cast to another pointer type first.

- **Example:**

c

int a = 10;

float b = 3.14;

void *vptr;

vptr = &a; *// Point to an integer*

printf("Value of a: %d\n", *((int*)vptr)); *// Cast to int* before dereferencing*

vptr = &b; *// Point to a float*

printf("Value of b: %f\n", *((float*)vptr)); *// Cast to float* before dereferencing*

---

4. What is linear array? Discuss the representation of linear array in memory? Write an algorithm for inserting an element into a liner array and deleting from an linear array

Linear Array: A linear array is a collection of similar data elements stored in contiguous memory locations. Each element is identified by an index or a subscript.

Memory Representation:
The elements of a linear array are stored in consecutive memory locations. If the base address of the array is B, the size of each element is S bytes, and the index starts from 0, then the address of the i-th element is:
Address(A[i]) = B + (i * S)

Algorithms:

- **Insertion:**

Algorithm: INSERT(LA, N, K, ITEM)

1. Set J = N // Initialize counter
2. Repeat Steps 3 and 4 while J >= K
3. text

Set `LA[J+1] = LA[J]` // Move element downward

4. text

Set `J = J - 1` // Decrement counter

5. Set LA[K] = ITEM // Insert new element
6. Set N = N + 1 // Reset N to new number of elements

- **Deletion:**

Algorithm: DELETE(LA, N, K, ITEM)

1. Set ITEM = LA[K] // Store the element to be deleted
2. Repeat for J = K to N-1
3. text

Set `LA[J] = LA[J+1]` // Move elements upward

4. Set N = N - 1 // Reset N to new number of elements

---

5. Differentiate between structures and unions. show examples for both

Feature	Structure (struct)	Union (union)
Memory	Each member has its own memory.	All members share the same memory.
Size	Sum of sizes of all members (plus padding).	Size of the largest member.
Access	All members can be accessed at any time.	Only one member can be used at a time.
Purpose	To group different data types together.	To provide memory efficiency for mutually exclu data.

Examples:

c

```c
// STRUCTURE Example
struct Student {
    int roll_no;
    char name[50];
    float marks;
}; // Size ~ sizeof(int) + 50*sizeof(char) + sizeof(float)

struct Student s1;
s1.roll_no = 1;
strcpy(s1.name, "Alice");
s1.marks = 85.5; // All members hold their values independently

// UNION Example
union Data {
    int i;
    float f;
    char str[20];
```

```
}; // Size = size of largest member (20 bytes for str[20])


union Data d;
d.i = 10;     // Now d holds an integer
printf("d.i : %d\n", d.i);
d.f = 220.5;  // Now d holds a float, the integer value is overwritten
printf("d.f : %f\n", d.f);
// Using d.i now would be incorrect and give garbage value.
```

---

6. How do you represent polynomial using array of structures and also write a function to add 2 polynomials

Representation:
A polynomial term can be represented by its coefficient and exponent. An array of such structures can represent the entire polynomial.

c

```
struct Term {
   int coeff;
   int exp;
};


struct Polynomial {
   struct Term terms[100]; // Array of terms
   int num_terms; // Number of non-zero terms
};
```

Example: Polynomial $3x^2 + 2x + 5$ would be:
terms[0] = {5, 0}, terms[1] = {2, 1}, terms[2] = {3, 2} and num_terms = 3.

Function to Add Two Polynomials:

c

```
void addPolynomials(struct Polynomial p1, struct Polynomial p2, struct Polynomial *p3) {
```

```
    int i = 0, j = 0, k = 0;

    while (i < p1.num_terms && j < p2.num_terms) {
        if (p1.terms[i].exp > p2.terms[j].exp) {
            p3->terms[k++] = p1.terms[i++];
        } else if (p1.terms[i].exp < p2.terms[j].exp) {
            p3->terms[k++] = p2.terms[j++];
        } else {
            // Exponents are equal, add coefficients
            p3->terms[k].coeff = p1.terms[i].coeff + p2.terms[j].coeff;
            p3->terms[k].exp = p1.terms[i].exp;
            // Only add if the sum is non-zero
            if (p3->terms[k].coeff != 0) k++;
            i++; j++;
        }
    }
    // Add remaining terms from p1
    while (i < p1.num_terms) {
        p3->terms[k++] = p1.terms[i++];
    }
    // Add remaining terms from p2
    while (j < p2.num_terms) {
        p3->terms[k++] = p2.terms[j++];
    }
    p3->num_terms = k;
}
```

---

7. Write the fast transpose algorithm to transpose the given sparse matrix. Express the sparse matrix as triplets and find its transpose

Sparse Matrix Representation (Triplets):
For a matrix with num non-zero elements, we use an array of triplets (row, col, value).

Given Matrix A:

$$A = \begin{bmatrix} 10 & 0 & 0 & 25 & 0 \\ 0 & 23 & 0 & 0 & 45 \\ 0 & 0 & 0 & 32 & 0 \\ 42 & 0 & 0 & 31 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 30 & 0 & 0 & 0 \end{bmatrix}$$

Triplet Representation of A:

Row	Col	Value
0	0	10
0	3	25
1	1	23
1	4	45
2	3	32
3	0	42
3	3	31
5	1	30

Fast Transpose Algorithm:
The algorithm uses two auxiliary arrays: row_terms[] (number of elements in each column of the original matrix, which becomes each row of the transpose) and starting_pos[] (starting position of each row in the transpose matrix).

Steps:

1. Find the number of elements in each column of the original matrix (row_terms for transpose).
2. Find the starting index in the transpose matrix for each row.
3. Iterate through the original triplet array and place each element in the correct position in the transpose triplet array.

Transpose of A (Triplet Representation):

Row	Col	Value
0	0	10
0	3	42
1	1	23
1	5	30
3	0	25
3	2	32
3	3	31
4	1	45

---

8. Write a C program to search for a pattern string in the given text string (KMP Algorithm)

c

```
#include <stdio.h>

#include <string.h>


void computeLPSArray(char* pat, int M, int* lps) {

    int len = 0;
```

```
    lps[0] = 0;
    int i = 1;
    while (i < M) {
        if (pat[i] == pat[len]) {
            len++;
            lps[i] = len;
            i++;
        } else {
            if (len != 0) {
                len = lps[len - 1];
            } else {
                lps[i] = 0;
                i++;
            }
        }
    }
}

void KMPSearch(char* pat, char* txt) {
    int M = strlen(pat);
    int N = strlen(txt);
    int lps[M];
    computeLPSArray(pat, M, lps);
    int i = 0; // index for txt[]
    int j = 0; // index for pat[]
    while (i < N) {
        if (pat[j] == txt[i]) {
            j++;
```

```
            i++;
        }
        if (j == M) {
            printf("Found pattern at index %d \n", i - j);
            j = lps[j - 1];
        } else if (i < N && pat[j] != txt[i]) {
            if (j != 0)
                j = lps[j - 1];
            else
                i = i + 1;
        }
    }
}

int main() {
    char txt[] = "ABABDABACDABABCABAB";
    char pat[] = "ABABCABAB";
    KMPSearch(pat, txt);
    return 0;
}
```

---

9. Consider a pattern p=ababab . construct the table and the corresponding labeled directed graph used in the fast or second pattern matching algorithm . Trace it for the input text T=abaaabababa

Pattern: p = "ababab"

1. Failure Function Table (KMP LPS Array):

Index	0	1	2	3	4	5
Char	a	b	a	b	a	b
LPS	0	0	1	2	3	4

2. Labeled Directed Graph (Finite Automaton):
This graph represents the state transitions. States are numbered 0 to 6 (length of pattern). State 6 is the accepting state.

- Start at state 0.
- On 'a': 0 -> 1
- On 'b': 0 -> 0
- On 'a': 1 -> 1? No, from state 1:
    - On 'a': 1 -> 1? (No, 'a' was expected for state 2). Actually, from state 1, on 'b': 1 -> 2. On 'a': 1 -> 1 (if no match, follow failure). This is complex to draw textually. The automaton is built such that from state i on seeing char c, you go to state lps[i-1] if there's a mismatch.

Tracing for T = "abaaababababa":
We'll simulate the KMP algorithm using the LPS table.
i (text index), j (pattern index)

- Start: i=0, j=0. T[0]='a', P[0]='a' -> match -> i=1, j=1.
- i=1, j=1. T[1]='b', P[1]='b' -> match -> i=2, j=2.
- i=2, j=2. T[2]='a', P[2]='a' -> match -> i=3, j=3.
- i=3, j=3. T[3]='a', P[3]='b' -> mismatch.
    - j = lps[2] = 1.
- i=3, j=1. T[3]='a', P[1]='b' -> mismatch.
    - j = lps[0] = 0.
- i=3, j=0. T[3]='a', P[0]='a' -> match -> i=4, j=1.
- i=4, j=1. T[4]='a', P[1]='b' -> mismatch.
    - j = lps[0] = 0.
- i=4, j=0. T[4]='a', P[0]='a' -> match -> i=5, j=1.

- i=5, j=1. T[5]='b', P[1]='b' -> match -> i=6, j=2.
- ... Continue this process...
- **Final Match:** The pattern will be found at the end when i=10 and j becomes 6.

---

10. Write a c program for the following stack operation on stack of integers 1. Push 2.pop 3. Display 4. Exit

c

```
#include <stdio.h>

#include <stdlib.h>

#define MAX 5


int stack[MAX];

int top = -1;


void push() {

   int item;

   if (top == MAX - 1) {

      printf("Stack Overflow! Cannot push.\n");

   } else {

      printf("Enter the element to push: ");

      scanf("%d", &item);

      top++;

      stack[top] = item;

      printf("Pushed %d onto the stack.\n", item);

   }

}


void pop() {
```

```
    if (top == -1) {
        printf("Stack Underflow! Cannot pop.\n");
    } else {
        printf("Popped element is %d.\n", stack[top]);
        top--;
    }
}

void display() {
    int i;
    if (top == -1) {
        printf("Stack is empty.\n");
    } else {
        printf("Stack elements are:\n");
        for (i = top; i >= 0; i--) {
            printf("%d\n", stack[i]);
        }
    }
}

int main() {
    int choice;
    while (1) {
        printf("\n*** Stack Menu ***\n");
        printf("1. Push\n2. Pop\n3. Display\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
```

```
        case 1: push(); break;
        case 2: pop(); break;
        case 3: display(); break;
        case 4: exit(0);
        default: printf("Invalid choice!\n");
      }
    }
    return 0;
}
```

---

11. Write an algorithm to evaluate postfix expression and apply the same for the given postfix expression ABC-D*+E$F+ and assume A=6, B=3, C=2, D=5, E=1 and F=7

Algorithm to Evaluate Postfix Expression:

1. Create an empty stack.
2. Scan the expression from left to right.
3. For each element:
   a. If it is an operand, push it onto the stack.
   b. If it is an operator, pop the required number of operands from the stack (two for binary operators), perform the operation, and push the result back onto the stack.
4. After the entire expression is scanned, the stack will contain one element, which is the final result.

Application: Expression: ABC-D*+E$F+
Given: A=6, B=3, C=2, D=5, E=1, F=7
Expression becomes: 6 3 2 - 5 * + 1 ^ 7 + (Assuming $ is exponentiation ^)

Symbol	Scanned	Operation	Stack (bottom -> top)	Explanation
A	6	Push(6)	6	
B	3	Push(3)	6, 3	

Symbol	Scanned	Operation	Stack (bottom -> top)	Explanation
C	2	Push(2)	6, 3, 2	
-	-	Pop()=2, Pop()=3	6	3 - 2 = 1
		Push(1)	6, 1	
D	5	Push(5)	6, 1, 5	
*	*	Pop()=5, Pop()=1	6	1 * 5 = 5
		Push(5)	6, 5	
+	+	Pop()=5, Pop()=6		6 + 5 = 11
		Push(11)	11	
E	1	Push(1)	11, 1	
$/^	^	Pop()=1, Pop()=11		11 ^ 1 = 11
		Push(11)	11	
F	7	Push(7)	11, 7	
+	+	Pop()=7, Pop()=11		11 + 7 = **18**
		Push(18)	**18**	**Final Result**

Result: 18

---

12. Outline the algorithm for infix to postfix. Convert the following infix expression to postfix using stack

Algorithm: Infix to Postfix Conversion

1. Initialize an empty stack and an empty output string.
2. Scan the infix expression from left to right.
3. For each element:
   a. **Operand:** Append it to the output string.
   b. **Left Parenthesis '(':** Push it onto the stack.
   c. **Right Parenthesis ')':** Pop from the stack and append to the output until a '(' is encountered. Pop and discard the '('.
   d. **Operator:**
   i. While the stack is not empty and the precedence of the scanned operator is less than or equal to the precedence of the operator at the top of the stack (and the top is not '('), pop the stack and append to the output.
   ii. Push the scanned operator onto the stack.
4. After the expression is scanned, pop all remaining operators from the stack and append them to the output.

Conversions:

1. **A+(B*C-(D/E^F)*G)*H**
   - **Postfix:** A B C * D E F ^ / G * - H * +
2. **(a+b)*d+c/f+c**
   - **Postfix:** a b + d * c f / + c +
3. **((H*((((A+(B+C)*D))F)*G)*E))+J)**
   - **Postfix:** H A B C + + D * F * G * E * * J +
4. **A$B*C-D+E|F|(G+H)**
   - Assuming $ = exponentiation (high precedence), | = logical OR (low precedence).
   - **Postfix:** A B $ C * D - E F + G H + | | (Precedence details needed for exact placement of |)
5. **A-B|(C-D$E)**
   - **Postfix:** A B C D E $ - | -
6. **(A+B*C)*(D+E-F)/J)**
   - **Postfix:** A B C * + D E + F - * J /
7. **((((a/b)-c)+(d*c))-a*c))**

- **Postfix:** a b / c - d c * + a c * -

8. **A-B|(C*D$E)**
    - **Postfix:** A B C D E $ * | -

Module-2

13. What are the disadvantages of ordinary queue how it overcome by circular queue? Discuss the implementation of circular queue using arrays? what are advantages of circular queue over ordinary queue

Disadvantages of Linear Queue:

- **Inefficient Memory Utilization:** In a linear array implementation, after a series of enqueue and dequeue operations, the "rear" pointer may reach the end of the array even if there is empty space at the beginning (a condition known as "False Overflow"). This wasted space cannot be reused.

How Circular Queue Overcomes This:
A circular queue treats the array as a circular structure. When the rear (or front) pointer reaches the end of the array, it wraps around to the beginning if there is space available. This allows for the efficient reuse of memory.

Implementation using Arrays:

c

```c
#define MAX 5

int cq[MAX];

int front = -1, rear = -1;


void enqueue(int item) {

   if ((front == 0 && rear == MAX - 1) || (front == rear + 1)) {

      printf("Queue Overflow\n");

      return;

   }

   if (front == -1) { // Queue is initially empty

      front = 0;
```

```
        rear = 0;
    } else {
        rear = (rear + 1) % MAX; // Wrap around
    }
    cq[rear] = item;
}

int dequeue() {
    if (front == -1) {
        printf("Queue Underflow\n");
        return -1;
    }
    int item = cq[front];
    if (front == rear) { // Queue has only one element
        front = -1;
        rear = -1;
    } else {
        front = (front + 1) % MAX; // Wrap around
    }
    return item;
}
```

Advantages of Circular Queue over Linear Queue:

- **Efficient Memory Use:** It reuses the empty spaces created by dequeue operations, preventing false overflow.
- **Better Performance:** More consistent performance for applications where elements are constantly added and removed.

---

14. How multiple stacks implemented using one dimensional array? Explain with suitable Example

Multiple stacks can be implemented in a single array by dividing the array into segments or by growing the stacks from opposite ends. The most common approach is **Two Stacks in a Single Array**.

Implementation: Two Stacks

- We start one stack from the left end (index 0) and the other from the right end (index MAX-1).
- Stack 1 grows towards the right (top1 increases).
- Stack 2 grows towards the left (top2 decreases).
- The stacks are full when top1 + 1 == top2.

Example Code:

c

```
#include <stdio.h>

#include <stdlib.h>

#define MAX 10


int arr[MAX];

int top1 = -1;

int top2 = MAX;


void push1(int data) {

   if (top1 < top2 - 1) {

      top1++;

      arr[top1] = data;

   } else {

      printf("Stack 1 Overflow\n");

   }

}


void push2(int data) {
```

```
    if (top1 < top2 - 1) {
        top2--;
        arr[top2] = data;
    } else {
        printf("Stack 2 Overflow\n");
    }
}

int pop1() {
    if (top1 >= 0) {
        int x = arr[top1];
        top1--;
        return x;
    } else {
        printf("Stack 1 Underflow\n");
        exit(1);
    }
}

int pop2() {
    if (top2 < MAX) {
        int x = arr[top2];
        top2++;
        return x;
    } else {
        printf("Stack 2 Underflow\n");
        exit(1);
    }
```

}

---

15. What is linked list? Explain different types of linked lists with a neat diagram

Definition: A linked list is a linear data structure where elements (called nodes) are stored at non-contiguous memory locations. Each node contains two parts:

1. **Data:** The value of the node.
2. **Pointer (or Link):** The address of the next node in the sequence.

Types of Linked Lists:

1. **Singly Linked List:**
   - Each node points only to the next node.
   - The last node points to NULL.
   - **Diagram:**
     [Data | Next] -> [Data | Next] -> [Data | Next] -> NULL
2. **Doubly Linked List:**
   - Each node has two pointers: one to the next node and one to the previous node.
   - Allows traversal in both directions.
   - **Diagram:**
     NULL <-> [Prev | Data | Next] <-> [Prev | Data | Next] <-> NULL
3. **Circular Linked List:**
   - The last node points back to the first node instead of NULL.
   - Can be singly circular or doubly circular.
   - **Singly Circular Diagram:**
     [Data | Next] -> [Data | Next] -> [Data | Next] --
     ^----------
4. **Circular Doubly Linked List:**
   - Combines features of both doubly and circular linked lists.
   - The next of the last node points to the head, and the prev of the head node points to the last node.

- **Diagram:**
  --[Prev|Data|Next] <-> [Prev|Data|Next] <-> [Prev|Data|Next]--
  |----------|

---

16. Write a C function to concatenate two singly linked list

c

```c
struct Node {
   int data;
   struct Node* next;
};

void concatenate(struct Node** head1, struct Node* head2) {
   struct Node* temp;
   // If the first list is empty, simply point head1 to head2
   if (*head1 == NULL) {
      *head1 = head2;
      return;
   }
   // Traverse to the end of the first list
   temp = *head1;
   while (temp->next != NULL) {
      temp = temp->next;
   }
   // Link the last node of the first list to the head of the second list
   temp->next = head2;
}
```

---

17. Give the structure definition of singly linked list. Write a C function to...

Structure Definition:

c

```
struct Node {
    int data;
    struct Node* next;
};
```

C Functions:

- **Delete a node at the beginning:**

c

```
void deleteAtBeginning(struct Node** head) {
    if (*head == NULL) {
        printf("List is empty.\n");
        return;
    }
    struct Node* temp = *head;
    *head = (*head)->next;
    free(temp);
}
```

- **Delete a node at the end:**

c

```
void deleteAtEnd(struct Node** head) {
    if (*head == NULL) {
        printf("List is empty.\n");
        return;
    }
    struct Node *temp = *head, *prev = NULL;
    // If only one node is present
    if (temp->next == NULL) {
        *head = NULL;
```

```
        free(temp);
        return;
    }
    // Traverse to the last node
    while (temp->next != NULL) {
        prev = temp;
        temp = temp->next;
    }
    prev->next = NULL;
    free(temp);
}
```

- **Inserting at beginning:**

c

```
void insertAtBeginning(struct Node** head, int new_data) {
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
    new_node->data = new_data;
    new_node->next = *head;
    *head = new_node;
}
```

- **Inserting at end:**

c

```
void insertAtEnd(struct Node** head, int new_data) {
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
    new_node->data = new_data;
    new_node->next = NULL;
    if (*head == NULL) {
        *head = new_node;
        return;
```

```
    }
    struct Node* temp = *head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = new_node;
}
```

- **Finding the length of the list:**

c

```
int findLength(struct Node* head) {
    int count = 0;
    struct Node* current = head;
    while (current != NULL) {
        count++;
        current = current->next;
    }
    return count;
}
```

- **Reverse a list:**

c

```
void reverseList(struct Node** head) {
    struct Node *prev = NULL, *current = *head, *next = NULL;
    while (current != NULL) {
        next = current->next; // Store next node
        current->next = prev; // Reverse current node's pointer
        prev = current;      // Move pointers one position ahead
        current = next;
    }
```

```
    *head = prev; // Reset head to the new first node
}
```

---

18. Develop a c function to implement various operations of queues using linked list

c

```c
#include <stdio.h>
#include <stdlib.h>

struct QNode {
    int key;
    struct QNode* next;
};

struct Queue {
    struct QNode *front, *rear;
};

struct QNode* newNode(int k) {
    struct QNode* temp = (struct QNode*)malloc(sizeof(struct QNode));
    temp->key = k;
    temp->next = NULL;
    return temp;
}

struct Queue* createQueue() {
    struct Queue* q = (struct Queue*)malloc(sizeof(struct Queue));
    q->front = q->rear = NULL;
    return q;
```

```
}

void enqueue(struct Queue* q, int k) {
    struct QNode* temp = newNode(k);
    // If queue is empty, new node is both front and rear
    if (q->rear == NULL) {
        q->front = q->rear = temp;
        return;
    }
    // Add the new node at the end and change rear
    q->rear->next = temp;
    q->rear = temp;
}

int dequeue(struct Queue* q) {
    if (q->front == NULL) {
        printf("Queue is empty\n");
        return -1;
    }
    // Store previous front and move front one node ahead
    struct QNode* temp = q->front;
    int item = temp->key;
    q->front = q->front->next;
    // If front becomes NULL, then change rear also to NULL
    if (q->front == NULL)
        q->rear = NULL;
    free(temp);
    return item;
```

```
}

int isEmpty(struct Queue* q) {
    return (q->front == NULL);
}
```

---

19. Write a c function to implement insert and delete operation on stack using linked list

c

```
#include <stdio.h>
#include <stdlib.h>

struct StackNode {
    int data;
    struct StackNode* next;
};

// Insert (Push) Operation
void push(struct StackNode** root, int data) {
    struct StackNode* stackNode = (struct StackNode*)malloc(sizeof(struct StackNode));
    stackNode->data = data;
    stackNode->next = *root;
    *root = stackNode;
    printf("%d pushed to stack\n", data);
}

// Delete (Pop) Operation
int pop(struct StackNode** root) {
    if (*root == NULL) {
```

```
        printf("Stack Underflow\n");
        return -1;
    }
    struct StackNode* temp = *root;
    *root = (*root)->next;
    int popped = temp->data;
    free(temp);
    return popped;
}
```

---

20. Develop a c program to add two polynomials using singly linked list

c

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int coeff;
    int exp;
    struct Node* next;
};

// Function to create a new node
void createNode(int coeff, int exp, struct Node** poly) {
    struct Node *temp = *poly, *new_node = (struct Node*)malloc(sizeof(struct Node));
    new_node->coeff = coeff;
    new_node->exp = exp;
    new_node->next = NULL;
```

```
    if (*poly == NULL) {
        *poly = new_node;
    } else {
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = new_node;
    }
}

// Function to add two polynomials
void polyAdd(struct Node* poly1, struct Node* poly2, struct Node** polySum) {
    struct Node *p1 = poly1, *p2 = poly2;

    while (p1 != NULL && p2 != NULL) {
        if (p1->exp > p2->exp) {
            createNode(p1->coeff, p1->exp, polySum);
            p1 = p1->next;
        } else if (p1->exp < p2->exp) {
            createNode(p2->coeff, p2->exp, polySum);
            p2 = p2->next;
        } else {
            createNode(p1->coeff + p2->coeff, p1->exp, polySum);
            p1 = p1->next;
            p2 = p2->next;
        }
    }
    // Add remaining terms of poly1
```

```
    while (p1 != NULL) {
        createNode(p1->coeff, p1->exp, polySum);
        p1 = p1->next;
    }
    // Add remaining terms of poly2
    while (p2 != NULL) {
        createNode(p2->coeff, p2->exp, polySum);
        p2 = p2->next;
    }
}

// Function to display a polynomial
void display(struct Node* poly) {
    if (poly == NULL) {
        printf("0\n");
        return;
    }
    while (poly != NULL) {
        printf("%dx^%d", poly->coeff, poly->exp);
        poly = poly->next;
        if (poly != NULL && poly->coeff >= 0) {
            printf(" + ");
        }
    }
    printf("\n");
}

int main() {
```

```
    struct Node *poly1 = NULL, *poly2 = NULL, *polySum = NULL;

    // Create first polynomial: 5x^2 + 4x^1 + 2x^0
    createNode(5, 2, &poly1);
    createNode(4, 1, &poly1);
    createNode(2, 0, &poly1);

    // Create second polynomial: -5x^1 + 5x^0
    createNode(-5, 1, &poly2);
    createNode(5, 0, &poly2);

    printf("First Polynomial: ");
    display(poly1);
    printf("Second Polynomial: ");
    display(poly2);

    polyAdd(poly1, poly2, &polySum);
    printf("Sum Polynomial: ");
    display(polySum);

    return 0;
}
```

Module - 3

21. What is circular doubly linked list. Write a c program to perform following operation on circular doubly linked list 1.insert a node at the beginning/rear end 2.Delete a node from the rear end 3. Search a node with given key value

**Circular

Continue